Lab 12: Lasso, Ridge, Trees, Neural Networks, and Word Vectors

SOCIOL 723, Social Statistics II, Thursday, November 12

Wenhao Jiang

Table of contents

1 Goals **2**

2 Data and the split **2**

3 Lasso, ridge, and elastic net **3**
- 3.1 Fit, choose λ, read the coefficients . . . 3
- 3.2 Prediction: the flexible design . . . 4
- 3.3 Choosing α for the elastic net by cross-validation . . . 4
- 3.4 Post-lasso . . . 5

4 Trees **6**

5 Random forests **7**
- 5.1 Tuning . . . 8
- 5.2 Variable importance . . . 8

6 Boosted trees **9**

7 Neural networks **10**
- 7.1 A deeper network with keras3 . . . 11

8 Word vectors **13**

9 Scoreboard **14**

10 Exercises **15**

11 Session information **15**

1 Goals

One task: predict log earnings in the GSS for people the model has not seen. For each method, the code you would run in your own work.

1. Penalized regression with `glmnet`: lasso, ridge, elastic net; λ by cross-validation; post-lasso.
2. Trees with `rpart`, random forests with `ranger` (tuning, variable importance), boosted trees with `xgboost`.
3. A neural network with `nnet`.
4. Word vectors with `word2vec`.
5. One scoreboard: every method's test R^2.

The test set is touched only to report the final number.

```
library(glmnet)       ## lasso, ridge, elastic net
library(rpart)        ## regression trees
library(rpart.plot)   ## drawing trees
library(ranger)       ## random forests
library(xgboost)      ## boosted trees
library(nnet)         ## one-hidden-layer neural networks
library(word2vec)     ## word vectors (skip-gram), no Python needed
library(janeaustenr)  ## the text of Jane Austen's six novels
library(plotmo)       ## plot_glmnet: labelled lasso paths
```

2 Data and the split

```
set.seed(723)
d <- readRDS("../Data/gss_earnings.rds")
## SEI and prestige are left out: both describe the respondent's occupation,
and SEI is
## built partly from occupational earnings, so they would leak the outcome
vars <- c("lnearn", "educ", "age", "female", "black", "otherace", "wordsum",
"pareduc",
          "hours", "childs", "evermar", "region", "year")
d <- d[complete.cases(d[, vars]), ]                  ## complete cases
d$region <- factor(d$region); d$yr <- factor(d$year)
## 23 main-effect columns
f_main <- lnearn ~ educ + age + female + black + otherace + wordsum + pareduc +
  hours + childs + evermar + region + yr
## a flexible design: all pairwise interactions plus squares
num <- c("educ", "age", "wordsum", "pareduc", "hours", "childs")
f_big <- as.formula(paste("lnearn ~ (educ + age + female + black + otherace +
wordsum + pareduc + hours + childs + evermar + region + yr)^2 +",
```

```
                          paste0("I(", num, "^2)", collapse = " + ")))
Xm <- model.matrix(f_main, d)[, -1]
Xb <- model.matrix(f_big, d)[, -1]
Xb <- Xb[, apply(Xb, 2, var) > 0]                    ## drop constant columns
Xb <- Xb[, !duplicated(t(Xb))]                       ## drop duplicated
columns
y <- d$lnearn; n <- nrow(d)
c(n = n, main = ncol(Xm), expanded = ncol(Xb))
#>        n     main expanded
#>     3509       23      243
## 30% test set, never used for fitting or tuning
te <- sample(n, round(0.3 * n)); tr <- setdiff(1:n, te)
## test R^2, with the training mean as the benchmark
r2 <- function(pred) 1 - mean((y[te] - pred)^2) / mean((y[te] - mean(y[tr]))^2)
## data frames for the tree, forest, and network functions
dtr <- data.frame(y = y[tr], Xm[tr, ]); dte <- data.frame(Xm[te, ])
```

3 Lasso, ridge, and elastic net

The lasso picks b to minimize

$$\sum_i \left(Y_i - b_0 - \sum_j X_{ij} b_j\right)^2 + \lambda \sum_j |b_j|.$$

Ridge uses $\lambda \sum_j b_j^2$; the elastic net mixes the two. In `glmnet`, `alpha = 1` is the lasso, `alpha = 0` ridge, anything in between an elastic net. `glmnet` divides the sum of squares by $2n$, so its `lambda` is the slides' $\lambda/(2n)$. It standardizes the columns before penalizing and reports coefficients on the original scale.

3.1 Fit, choose λ, read the coefficients

`cv.glmnet` fits the whole path of λ values and runs K-fold cross-validation on the training cases. `lambda.min` has the smallest CV error; `lambda.1se` is the largest λ within one standard error of it (a simpler model that predicts about as well).

```
set.seed(7231)
cl_m <- cv.glmnet(Xm[tr, ], y[tr], alpha = 1, nfolds = 10)   ## 23 main
effects
## coefficients at lambda.1se: the variables kept and their sizes
b <- coef(cl_m, s = "lambda.1se")[-1, 1]
round(b[b != 0], 3)
#>    educ     age  female   black wordsum pareduc   hours evermar  yr2022
#>   0.088   0.009  -0.231  -0.036   0.022   0.005   0.004   0.121   0.055
```

```
par(mfrow = c(1, 2), cex = 0.8)
## coefficient paths: one line per covariate, the 8 largest named, vertical
line at lambda.1se
plot_glmnet(cl_m$glmnet.fit, s = cl_m$lambda.1se, label = 8)
plot(cl_m)                                              ## CV error with +/- 1
s.e. bars
```

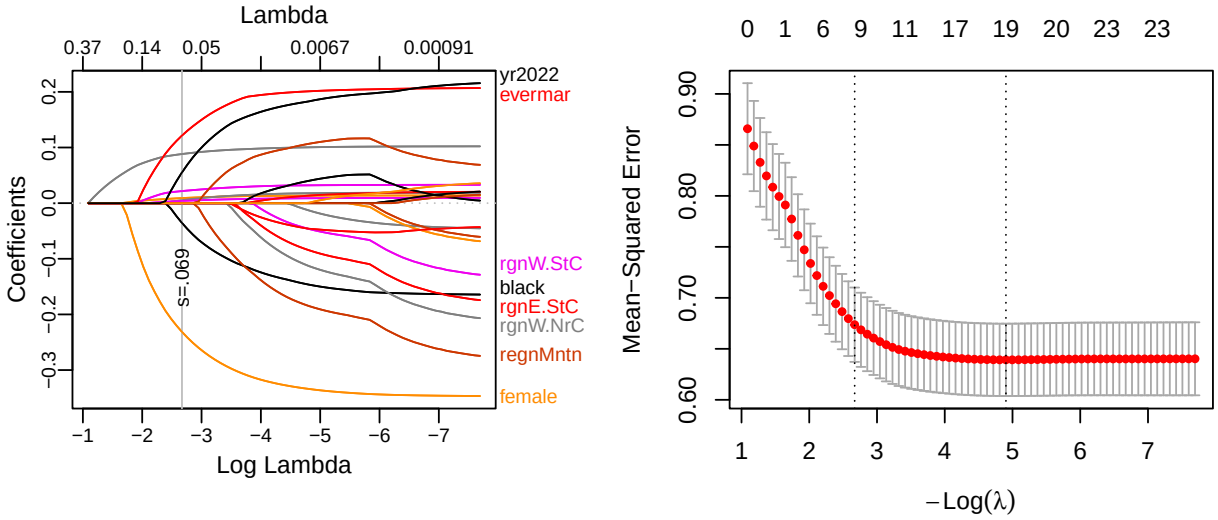


Figure 1: Lasso on the 23 main effects. Left: each covariate's coefficient path as the penalty shrinks; the eight largest named; vertical line at lambda.1se. Right: 10-fold cross-validated MSE with 1-s.e. bars; dashed lines at lambda.min and lambda.1se; top axis = number of variables kept.

3.2 Prediction: the flexible design

The same code on the 243-term design, then ridge and the elastic net:

```
cl <- cv.glmnet(Xb[tr, ], y[tr], alpha = 1, nfolds = 10)     ## lasso
cr <- cv.glmnet(Xb[tr, ], y[tr], alpha = 0, nfolds = 10)     ## ridge
ce <- cv.glmnet(Xb[tr, ], y[tr], alpha = 0.5, nfolds = 10)   ## elastic net
c(lasso_terms_kept = unname(cl$nzero[cl$lambda == cl$lambda.min]),
  lasso = r2(predict(cl, Xb[te, ], s = "lambda.min")),
  ridge = r2(predict(cr, Xb[te, ], s = "lambda.min")),
  elastic_net = r2(predict(ce, Xb[te, ], s = "lambda.min")))
#> lasso_terms_kept            lasso            ridge      elastic_net
#>          45.0000           0.2611           0.2515           0.2615
```

3.3 Choosing α for the elastic net by cross-validation

The elastic-net penalty is $\lambda\big[\frac{1-\alpha}{2}\sum_j b_j^2 + \alpha\sum_j |b_j|\big]$: $\alpha = 0$ is ridge, $\alpha = 1$ the lasso. `cv.glmnet` tunes λ for one α, so loop over a grid of α and keep the pair (α, λ) with the smallest CV error.

Use the **same folds** for every α (`foldid`), so the comparison is not driven by different splits.

```
set.seed(723)
foldid <- sample(rep(1:10, length.out = length(tr)))   ## one fold assignment
for all alphas
alphas <- seq(0, 1, by = 0.1)
fits <- lapply(alphas, function(a) cv.glmnet(Xb[tr, ], y[tr], alpha = a, foldid
= foldid))
cv_min <- sapply(fits, function(f) min(f$cvm))         ## best CV error for
each alpha
data.frame(alpha = alphas, cv_mse = round(cv_min, 4))
#>    alpha cv_mse
#> 1    0.0 0.6500
#> 2    0.1 0.6445
#> 3    0.2 0.6445
#> 4    0.3 0.6448
#> 5    0.4 0.6448
#> 6    0.5 0.6449
#> 7    0.6 0.6448
#> 8    0.7 0.6449
#> 9    0.8 0.6450
#> 10   0.9 0.6451
#> 11   1.0 0.6451
best <- which.min(cv_min)
ce_best <- fits[[best]]
c(alpha = alphas[best], lambda = ce_best$lambda.min,
  terms_kept = unname(ce_best$nzero[ce_best$lambda == ce_best$lambda.min]),
  test_R2 = r2(predict(ce_best, Xb[te, ], s = "lambda.min")))
#>      alpha     lambda terms_kept    test_R2
#>     0.1000     0.1958    67.0000     0.2613
```

The CV errors across α are usually close: once λ is tuned, the mix matters less than the amount of shrinkage. The test set is used once, for the chosen pair.

3.4 Post-lasso

Refit OLS on the variables the lasso kept: lasso chooses *which*, OLS chooses *how large*.

```
keep <- names(b)[b != 0]                              ## kept at lambda.1se,
main effects
post <- lm(y[tr] ~ Xm[tr, keep])
round(rbind(lasso = b[keep], post_lasso = coef(post)[-1]), 3)
#>             educ   age female  black wordsum pareduc hours evermar yr2022
#> lasso      0.088 0.009 -0.231 -0.036   0.022   0.005 0.004   0.121  0.055
```

```
#> post_lasso 0.102 0.014 -0.348 -0.160   0.032   0.019 0.009   0.221  0.216
```

4 Trees

A regression tree splits the data in two at the cut (j, s) that makes $\sum_{i \in L}(Y_i - \bar{Y}_L)^2 + \sum_{i \in R}(Y_i - \bar{Y}_R)^2$ smallest, then splits each part again; each leaf predicts its average.

```
t3 <- rpart(y ~ ., data = dtr,  ## y ~ . : predict y from every other column
            control = rpart.control(
              maxdepth = 3,  ## at most 3 splits from the root to a leaf
              (default 30)
              cp = 0,  ## complexity penalty: keep a split only if it cuts the
    ##   error by at least cp x (root error); default 0.01
              minsplit = 20,  ## a node needs at least 20 cases to be split
              (default)
              minbucket = 7))  ## every leaf keeps at least 7 cases (default
              minsplit/3)
```

```
rpart.plot(t3, type = 2,  ## split label under each node
           extra = 1,  ## print the number of cases in each node
           under = TRUE, fallen.leaves = TRUE, cex = 0.6)
```

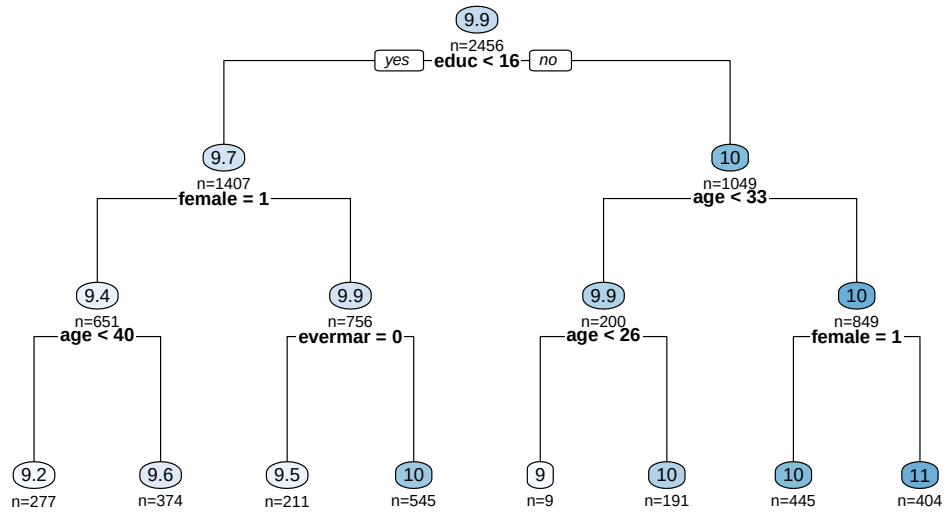


Figure 2: The depth-3 tree on the GSS. Below 16 years of schooling the tree splits next on sex; above, on age: an interaction.

The usual workflow: grow a large tree, then prune it back by cross-validation. `rpart` cross-validates the pruning sequence as it grows (10 folds by default); keep the complexity penalty `cp` with the smallest CV error.

```r
set.seed(723)
big <- rpart(y ~ ., data = dtr,
             cp = 0.001,  ## small penalty: grow a large tree
             xval = 10)  ## 10-fold CV of every pruned size (the default)
round(big$cptable[1:6, ], 4)  ## CP, splits, training error, CV error (xerror)
and its s.e.
#>       CP nsplit rel error xerror   xstd
#> 1 0.1216      0    1.0000 1.0005 0.0468
#> 2 0.0300      1    0.8784 0.8795 0.0429
#> 3 0.0170      2    0.8484 0.8570 0.0424
#> 4 0.0159      3    0.8314 0.8507 0.0425
#> 5 0.0135      4    0.8155 0.8355 0.0422
#> 6 0.0135      5    0.8019 0.8257 0.0418
cp_best <- big$cptable[which.min(big$cptable[, "xerror"]), "CP"]
pruned <- prune(big, cp = cp_best)  ## cut back every split worth less than
cp_best
c(cp = cp_best, leaves = sum(pruned$frame$var == "<leaf>"), test_R2 =
r2(predict(pruned, dte)))
#>        cp    leaves   test_R2
#>  0.005172 13.000000  0.217657
```

5 Random forests

Grow many deep trees on bootstrap samples; at each split try only `mtry` random predictors (default: $\sqrt{p}$ rounded down, here 4); average the trees.

```r
rf <- ranger(y ~ ., data = dtr,
             num.trees = 1000,  ## trees to average (default 500); more never
             hurts
             mtry = 4,  ## predictors tried at each split; default
             floor(sqrt(p))
             min.node.size = 5,  ## smallest leaf; default 5 for regression, 1
             for classes
             replace = TRUE,  ## bootstrap samples (default); FALSE =
             subsamples
             importance = "permutation",  ## compute permutation importance
             (default "none")
             seed = 723)  ## ranger's own seed: same forest every run
c(oob_mse = rf$prediction.error,  ## out-of-bag: a free estimate of test MSE
  test_mse = mean((y[te] - predict(rf, dte)$predictions)^2),
  test_R2 = r2(predict(rf, dte)$predictions))
#>  oob_mse test_mse  test_R2
```

```
#>   0.6256   0.6451   0.2722

## Random Forests build each tree using a bootstrap sample of the data
## (leaving out roughly 36.8% of observations per tree), ranger automatically
## evaluates prediction error on those left-out samples without needing
separate k-fold splits
```

5.1 Tuning

Judge `mtry` and `min.node.size` (smallest leaf) by the out-of-bag MSE.

```
M <- c(1, 3, 5, 8, 23); N <- c(1, 5, 20, 50, 100)
oob <- sapply(N, function(nd) sapply(M, function(m)
  ranger(y ~ ., dtr, num.trees = 1000, mtry = m, min.node.size = nd, seed =
  723)$prediction.error))
dimnames(oob) <- list(paste0("mtry=", M), paste0("node=", N))
round(oob, 3)
#>         node=1 node=5 node=20 node=50 node=100
#> mtry=1   0.740  0.740   0.742   0.744    0.749
#> mtry=3   0.630  0.629   0.629   0.638    0.648
#> mtry=5   0.626  0.623   0.623   0.625    0.631
#> mtry=8   0.633  0.629   0.626   0.623    0.627
#> mtry=23  0.663  0.656   0.642   0.635    0.634
rf_tun <- ranger(y ~ ., dtr, num.trees = 1000, mtry = 8, min.node.size = 50,
seed = 723)
c(default = r2(predict(rf, dte)$predictions), tuned = r2(predict(rf_tun,
dte)$predictions))
#> default   tuned
#>  0.2722  0.2860
```

5.2 Variable importance

Permutation importance: shuffle one column, predict again, record how much the error rises (`ranger` does this on each tree's out-of-bag cases). **Impurity importance**: add up how much each variable's splits cut the sum of squares. Ask for permutation importance explicitly; it is not the default.

```
rf_i <- ranger(y ~ ., dtr, num.trees = 1000, importance = "impurity", seed =
723)
ip <- sort(rf$variable.importance, decreasing = TRUE)[1:10]
ii <- sort(rf_i$variable.importance / sum(rf_i$variable.importance), decreasing
= TRUE)[1:10]
par(mfrow = c(1, 2), mar = c(4, 7, 2, 1), cex = 0.8)
```

```
barplot(rev(ip), horiz = TRUE, las = 1, col = "navy", border = NA,
        main = "permutation", xlab = "rise in OOB MSE")
barplot(rev(ii), horiz = TRUE, las = 1, col = "firebrick", border = NA,
        main = "impurity", xlab = "share of SSE reduction")
```

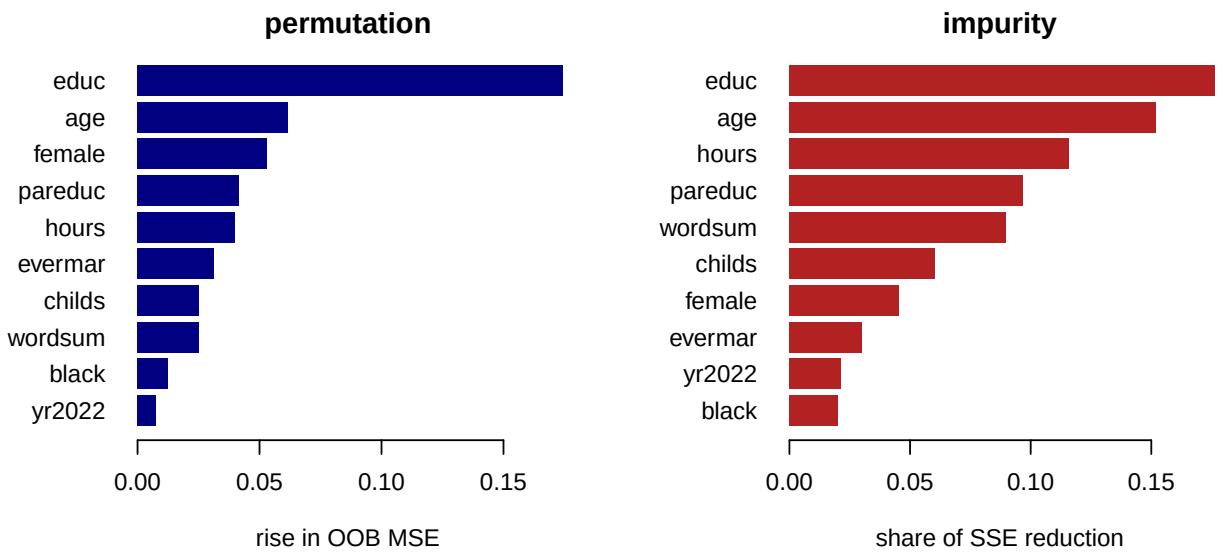


Figure 3: Variable importance for the random forest, top 10. Left: permutation (rise in out-of-bag MSE when shuffled). Right: impurity (share of all SSE reduction).

Correlated variables share credit, and impurity importance favours variables with many distinct values. Importance describes the forest's predictions, not causes.

6 Boosted trees

Start from the mean, fit a small tree to the residuals, add a fraction η (the learning rate) of it, and repeat: $\hat{f} \leftarrow \hat{f} + \eta\, \hat{f}_m$. `xgb.cv` picks the number of steps by 5-fold CV, stopping once 50 more steps bring no improvement; `xgb.train` then fits that many steps on all training cases.

```
set.seed(723)
dm <- xgb.DMatrix(Xm[tr, ], label = y[tr])
pars <- list(objective = "reg:squarederror",  ## squared error;
"binary:logistic" for 0/1
             eta = 0.05,  ## learning rate: share of each tree added (default
             0.3)
             max_depth = 3,  ## depth of each small tree (default 6)
             min_child_weight = 1  ## smallest leaf, in cases for squared error
             (default 1)
             )
cvb <- xgb.cv(params = pars, data = dm,
```

```
              nrounds = 2000,  ## maximum number of trees
              nfold = 5,  ## 5-fold CV inside the training set
              early_stopping_rounds = 50,  ## stop once 50 more trees bring no
              CV gain
              verbose = 0)
steps <- which.min(cvb$evaluation_log$test_rmse_mean)  ## steps with the lowest
CV error
bst <- xgb.train(params = pars, data = dm, nrounds = steps, verbose = 0)  ##
refit on all training cases
c(steps = steps, test_R2 = r2(predict(bst, Xm[te, ])))
#>    steps  test_R2
#> 120.0000   0.2856
```

7 Neural networks

One hidden layer: $A_k = \sigma(w_{k0} + \sum_j w_{kj} X_j)$, then $f(X) = \beta_0 + \sum_k \beta_k A_k$, fit by gradient descent. `size` = number of hidden units; `decay` = weight decay (a ridge penalty on the weights).

```
## raw X and raw y, no rescaling (see the note below)
## 2 hidden units and weight decay 0.1 (chosen by CV in figs/w12_gss.R)
## average 5 random starts: the loss is not convex
set.seed(723)
nn_pred <- rowMeans(replicate(5, {
  m <- nnet(Xm[tr, ], y[tr],
            size = 2,  ## hidden units in the one hidden layer
            decay = 0.1,  ## weight decay: ridge penalty on all weights
            (default 0)
            linout = TRUE,  ## linear output for a continuous y (default:
            logistic)
            maxit = 500,  ## at most 500 optimizer iterations (default 100)
            trace = FALSE)  ## do not print the optimizer's progress
  as.vector(predict(m, Xm[te, ]))
}))
c(parameters = (ncol(Xm) + 1) * 2 + 2 + 1, test_R2 = r2(nn_pred))
#> parameters    test_R2
#>    51.0000     0.2608
```

`nnet` works on the raw variables here because the weight decay keeps every weight small. Set `decay = 0` and the fit gets much worse (Exercise 6). The usual safe habit is still to put the inputs on one scale, as in the next model.

7.1 A deeper network with keras3

`keras3` drives Keras/TensorFlow from R. The first call downloads a private Python and TensorFlow (about a minute, once); after that it runs like any package.

The model has two hidden layers of ReLU units, $23 \to 32 \to 16 \to 1$, with dropout after each, trained by Adam (a version of stochastic gradient descent) on mini-batches, with 20% of the training cases held back to decide when to stop.

```
library(keras3)
set_random_seed(723)  ## seeds R, Python, and TensorFlow together

## 0. Rescale. Keras starts every weight near 0 and Adam takes small steps, so
##    (a) inputs on very different scales (age ~ 45, female 0/1) slow training,
and
##    (b) predictions start near 0 while log earnings sit near 10.
##    So: inputs to mean 0, sd 1, and the outcome too; undo it at the end.
##    Means and sds come from the TRAINING cases only.
mu <- colMeans(Xm[tr, ]); sdv <- apply(Xm[tr, ], 2, sd)
Ztr <- scale(Xm[tr, ], mu, sdv); Zte <- scale(Xm[te, ], mu, sdv)
ym <- mean(y[tr]); ysd <- sd(y[tr])

## 1. Describe the network, layer by layer. The pipe |> adds one layer at a
time.
model <- keras_model_sequential(input_shape = ncol(Ztr)) |>  ## 23 standardized
inputs
  layer_dense(units = 32,  ## hidden layer 1: 32 units
              activation = "relu") |>  ##   each unit: max(0, w0 + sum_j w_j
              X_j)
  layer_dropout(rate = 0.2) |>  ## in training, switch off a random 20% of the
    ##   units at each step (against overfitting)
  layer_dense(units = 16, activation = "relu") |>  ## hidden layer 2: 16 units
  layer_dropout(rate = 0.2) |>
  layer_dense(units = 1)  ## output: one number, no activation (= linear)
## weights in each layer: (23 + 1) x 32, 0, (32 + 1) x 16, 0, (16 + 1) x 1
## (print `model` in the console for Keras's own summary table)
sapply(model$layers, function(layer) layer$count_params())
#> [1] 768   0 528   0  17

## 2. Choose the loss and the optimizer.
model |> compile(
  loss = "mse",  ## mean squared error; "binary_crossentropy" for 0/1
  optimizer = optimizer_adam(learning_rate = 0.001))  ## Adam; 0.001 is its
  default step size
```

```
## 3. Train. fit() changes `model` in place and returns the loss history.
##    validation_split takes the LAST 20% of rows, so shuffle the rows first
shuf <- sample(nrow(Ztr))
history <- model |> fit(
  x = Ztr[shuf, ], y = ((y[tr] - ym) / ysd)[shuf],  ## standardized inputs and
  outcome
  epochs = 200,  ## at most 200 passes through the training data
  batch_size = 32,  ## one gradient step per mini-batch of 32 cases
  validation_split = 0.2,  ## hold out the last 20% of rows to monitor
  callbacks = list(callback_early_stopping(
    monitor = "val_loss",  ## watch the held-out loss
    patience = 10,  ## stop after 10 epochs without improvement
    restore_best_weights = TRUE)),  ## and go back to the best epoch's weights
  verbose = 0)  ## no progress bar

## 4. Predict, and undo the standardization of y.
keras_pred <- ym + ysd * as.vector(predict(model, Zte, verbose = 0))
c(epochs_run = length(history$metrics$loss), test_R2 = r2(keras_pred))
#> epochs_run    test_R2
#>    34.0000     0.2281
```

```
plot(history)  ## loss by epoch: training vs. validation
```

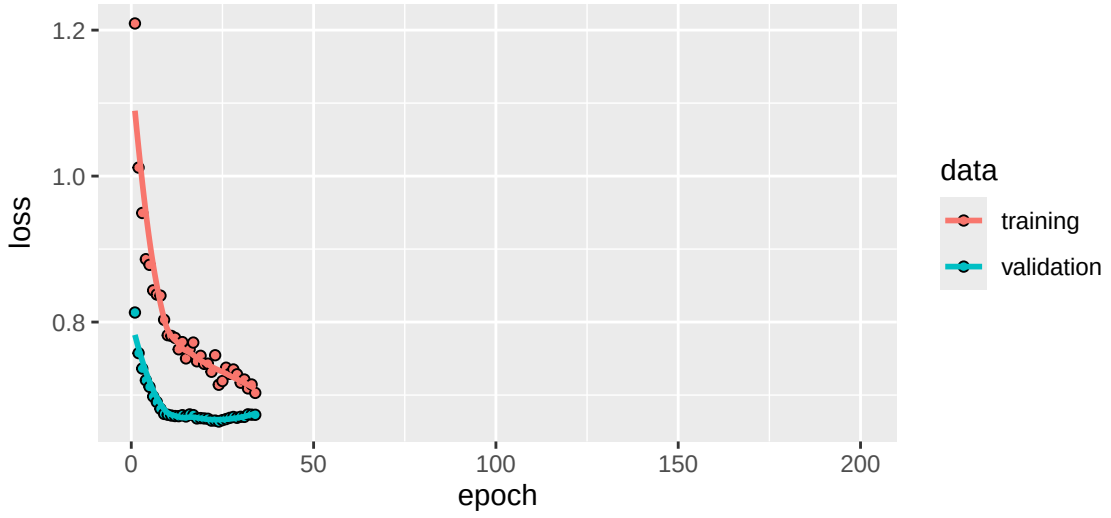


Figure 4: Training and validation loss by epoch. Early stopping ends training once the validation loss stops falling.

With 2,456 cases and 23 survey variables, the deep network does no better than the two-unit `nnet`: deep networks pay off with many raw inputs (pixels, words), not here.

8 Word vectors

Skip-gram word2vec predicts each word's neighbours, $\Pr(c \mid w) = \exp\left(\sum_k u_{wk} v_{ck}\right) / \sum_{c'} \exp\left(\sum_k u_{wk} v_{c'k}\right)$; the fitted u_w is word w's vector. Similarity is the cosine, $\sum_k a_k b_k / (\sqrt{\sum_k a_k^2}\sqrt{\sum_k b_k^2})$.

The text: Jane Austen's six novels, about 730,000 words. The `word2vec` package does not take a seed, so the numbers change a little from run to run; the patterns do not.

```
txt <- tolower(austen_books()$text)
txt <- gsub("[^a-z ]", " ", txt[nchar(txt) > 0])  ## letters only
m <- word2vec(txt,
              type = "skip-gram",  ## predict neighbours from the word ("cbow":
              the reverse)
              dim = 50,  ## numbers per word, K (default 50)
              window = 5,  ## neighbours = words within 5 positions (default
              5)
              iter = 20,  ## passes through the text (default 5)
              negative = 5,  ## fake pairs per real pair, negative sampling
              (default 5)
              min_count = 5,  ## drop words seen fewer than 5 times (default
              5)
              threads = 1)  ## one thread: fewer run-to-run differences
emb <- as.matrix(m)  ## one row per word
dim(emb)
#> [1] 5762   50
## nearest words by cosine
predict(m, c("sister", "letter"), type = "nearest", top_n = 5)
#> $sister
#>    term1   term2 similarity rank
#> 1 sister brother     0.9356    1
#> 2 sister  mother     0.9072    2
#> 3 sister  father     0.9041    3
#> 4 sister  cousin     0.8826    4
#> 5 sister    aunt     0.8405    5
#>
#> $letter
#>    term1    term2 similarity rank
#> 1 letter     note     0.8930    1
#> 2 letter sentence     0.8399    2
#> 3 letter  written     0.8009    3
#> 4 letter  chapter     0.7964    4
#> 5 letter  account     0.7953    5
## arithmetic: brother - he + she
predict(m, emb["brother", ] - emb["he", ] + emb["she", ], type = "nearest",
top_n = 5)
```

```
#>      term similarity rank
#> 1 brother     0.9148    1
#> 2  sister     0.9085    2
#> 3  mother     0.8310    3
#> 4   niece     0.8295    4
#> 5    aunt     0.8053    5
```

A cultural dimension, as in Kozlowski, Taddy, and Evans (2019): average several male-minus-female differences into one direction, then take each word's cosine with it (positive = closer to the male side of Austen's usage).

```
cosine <- function(a, b) sum(a * b) / sqrt(sum(a^2) * sum(b^2))
gender <- (emb["he", ] - emb["she", ] + emb["man", ] - emb["woman", ] +
           emb["his", ] - emb["her", ] + emb["mr", ] - emb["mrs", ]) / 4
ws <- c("captain", "clergyman", "estate", "fortune", "business", "marriage",
        "beauty", "music", "needlework", "lady")
round(sort(sapply(ws, function(w) cosine(emb[w, ], gender))), 2)
#>       lady      music     beauty needlework   marriage    captain
#>      -0.32      -0.12      -0.08      -0.07       0.10       0.19
#>    fortune   business     estate  clergyman
#>       0.21       0.22       0.33       0.37
```

Pretrained transformers (BERT, GPT-style models) run through Python. The `text` package wraps them; shown but not run:

```
library(text)  ## textrpp_install() once: sets up Python and the transformers
library
e <- textEmbed(c("The survey was fielded in 2022.", "The river bank flooded."),
               model = "bert-base-uncased")
dim(e$texts$texts)  ## one row of 768 numbers per text: use them as X in a
ridge or logit
```

9 Scoreboard

```
ols <- lm.fit(cbind(1, Xm[tr, ]), y[tr])
sort(c(OLS = r2(as.vector(cbind(1, Xm[te, ]) %*% coef(ols))),
       lasso = r2(predict(cl, Xb[te, ], s = "lambda.min")),
       ridge = r2(predict(cr, Xb[te, ], s = "lambda.min")),
       elastic_net = r2(predict(ce, Xb[te, ], s = "lambda.min")),
       pruned_tree = r2(predict(pruned, dte)),
       random_forest = r2(predict(rf, dte)$predictions),
       boosting = r2(predict(bst, Xm[te, ])),
```

```
       neural_net = r2(nn_pred),
       deep_net_keras = r2(keras_pred)), decreasing = TRUE) |> round(3)
#>       boosting  random_forest    elastic_net          lasso     neural_net
#>          0.286          0.272          0.261          0.261          0.261
#>          ridge            OLS deep_net_keras    pruned_tree
#>          0.251          0.251          0.228          0.218
```

10 Exercises

1. **Two rules for λ.** Print the main-effect lasso coefficients at `lambda.min` and at `lambda.1se`. Which variables differ? Which model predicts better on the test set?
2. **Post-lasso.** Why is every post-lasso coefficient larger in size than the lasso one? When would post-lasso predict worse than lasso?
3. **Pruning.** Grow `big` with `cp = 0.0001` instead. Does the pruned tree change?
4. **Forests.** Add `mtry` = 2 and 12 to the `rf-tune` grid, and try `num.trees` = 50. Report the out-of-bag MSE.
5. **Boosting's learning rate.** In the `xgboost` chunk, try `eta` = 0.01, 0.1, 0.5. How many steps does CV choose for each, and which predicts best?
6. **Scaling and weight decay.** Rerun `nn-gss` with `decay = 0`, then with `Ztr`, `Zte` (from the `keras` chunk) as inputs. When does rescaling matter?
7. **Word vectors.** In `w2v-austen`, change `window` to 2 and then 10. How do the nearest words to *letter* change? Build a *rich* − *poor* dimension and place five words on it.
8. **Prediction is not inference.** Take the post-lasso coefficient on `female`. Explain in two sentences why you would not report it with the usual standard error.

11 Session information

```
## R version and the versions of the packages loaded above
cat(R.version.string, "\n")
#> R version 4.5.2 (2025-10-31)
sapply(sessionInfo()$otherPkgs, `[[`, "Version")
#>      keras3      plotmo     plotrix     Formula janeaustenr    word2vec
#>     "1.5.1"     "3.7.1"    "3.8-14"     "1.2-5"     "1.0.0"     "0.4.1"
#>        nnet     xgboost      ranger  rpart.plot       rpart      glmnet
#>    "7.3-20"   "3.2.1.1"    "0.18.0"     "3.1.5"    "4.1.24"       "5.1"
#>      Matrix
#>     "1.7-4"
```